# Урок №25

## Хранение данных в настройках приложения

## Цель работы

Научиться сохранять, получать и удалять данные в Android-приложении с помощью `SharedPreferences`.

## О чем урок

Представьте, что пользователь вводит свое имя в приложение. **Если закрыть приложение** и открыть его снова, **введенное имя может пропасть.**

Чтобы этого не происходило, можно сохранить имя на устройстве и восстановить его при следующем запуске.

Для этого используется `SharedPreferences` - встроенный механизм Android для хранения данных в формате "ключ - значение".

Например:

| Ключ | Значение |
|---|---|
| `user_name` | "Фарид" |
| `rating` | 67 |
| `is_boss` | false |

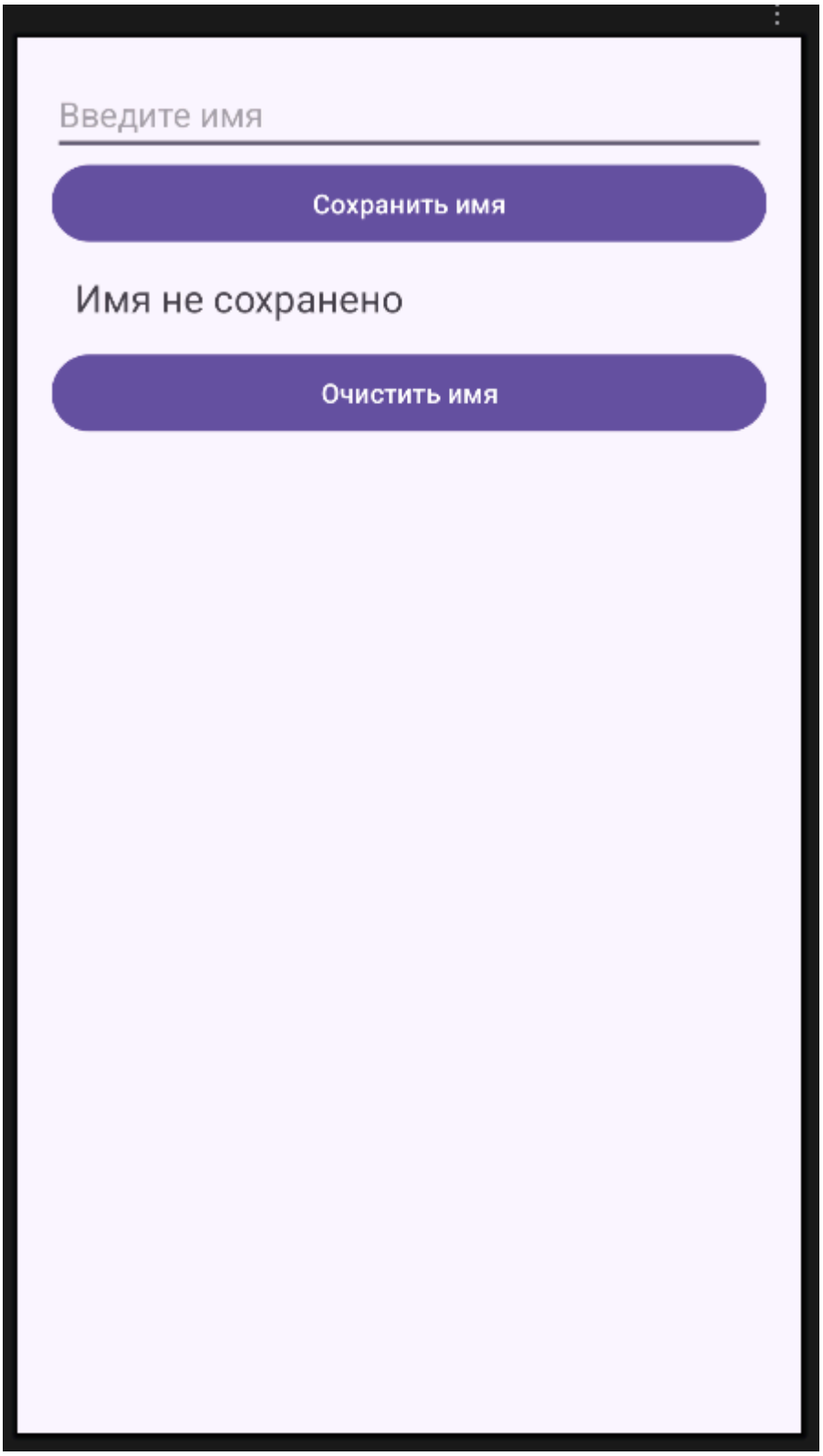
## Задание

Создадим приложение, которое позволяет пользователю:

1. Ввести свое имя
2. Сохранить имя на устройстве
3. Восстановить имя после повторного запуска приложения
4. Очистить сохраненное имя

## Часть 1. Создание интерфейса

Откройте файл `activity_main.xml` и разместите на экране поле для ввода имени, две кнопки и текст для отображения результата:




## Часть 2. Подготовка MainActivity

Откройте файл `MainActivity.java`. Сначала объявим **переменные** для хранилища и элементов интерфейса:

```
private EditText nameEditText;
private Button saveButton;
private Button clearButton;
private TextView nameTextView;
```

```
private SharedPreferences preferences;
```

`nameEditText`, `saveButton`, `clearButton` и `nameTextView` заполните в `onCreate` при помощи `findViewById()`.

А `preferences` заполните таким образом:

```
preferences = getSharedPreferences(
    "user_settings",
    Context.MODE_PRIVATE
);
```

Разберем создание хранилища:

- `getSharedPreferences()` - получает доступ к хранилищу
- `user_settings` - имя хранилища
- `MODE_PRIVATE` - режим, при котором хранилище доступно **только нашему приложению**

## Часть 3. Сохранение имени

Для сохранения данных используется `.edit()`. Он позволяет изменять содержимое хранилища.

Добавьте внутри метода `onCreate()` обработчик `setOnClickListener(...)` для `saveButton`, внутри него должен быть такой код:

```
// получаем имя из поля для ввода.
// .trim() удаляет лишние проблеы
String name = nameEditText.getText().toString().trim();

// запускаем редактирование при помощи .edit()
preferences.edit()
    .putString("user_name", name)   // помещаем в ячейку "user_name" наше имя
    .apply();   // сохраняем

if (name.isEmpty()) {
    nameTextView.setText("Имя не сохранено");
} else {
    nameTextView.setText("Сохраненное имя: " + name);
}
```

Основные команды:

- `getText().toString()` - получает текст из поля ввода
- `putString("user_name", name)` - сохраняет строку под ключом `user_name`
- `apply()` - применяет изменения и сохраняет

> **Проверьте работу:** введите имя и нажмите кнопку "Сохранить имя".

## Часть 4. Восстановление сохраненного имени

Чтобы имя **автоматически** отображалось *после повторного запуска приложения*, необходимо **прочитать** его из хранилища.

Добавьте в конец метода `onCreate()`:

```
// пробуем достать из хранилища строку из ячейки "user_name"
// если ячейка будет не существовать, то получим - ""
String savedName = preferences.getString("user_name", "");

// проверяем не пустое ли сохраненное имя
if (!savedName.isEmpty()) {
    // и если нет, то заполняем наш интерфейс сохраненной инфой
    nameEditText.setText(savedName);
    nameTextView.setText("Сохраненное имя: " + savedName);
}
```

Метод `getString()` возвращает сохраненную строку. Если значение отсутствует, возвращается пустая строка, указанная вторым аргументом.

> **Проверьте работу:**
>
> 1. Введите имя и сохраните его
> 2. Остановите приложение
> 3. Запустите его повторно
> 4. Убедитесь, что **имя восстановилось**

## Часть 5. Удаление сохраненного имени

Добавим обработчик кнопки "Очистить имя".

В конец метода `onCreate()` добавьте обработчик `setOnClickListener(...)` для `clearButton`, который будет содеражать такой код:

```
// запускаем редактирование при помощи .edit()
preferences.edit()
    .remove("user_name")    // удаляем ячейку "user_name"
    .apply();   // сохраняем

// не забываем так же обновить интерфейс
nameEditText.setText("");
nameTextView.setText("Имя не сохранено");
```

Метод `remove()` удаляет значение по указанному ключу.

Также существует метод `clear()`, который удаляет **все данные** из выбранного хранилища. В нашей работе используем `remove()`, поскольку нужно удалить только имя пользователя.

> **Проверьте работу:** сохраните имя, очистите его и повторно запустите приложение. Старое имя **не должно восстановиться**.

## Самостоятельное задание

Доработайте приложение, чтобы оно сохраняло не только имя, но и возраст пользователя.

**Необходимо**:

1. Добавить поле для ввода возраста
2. Реализовать сохранение имени и возраста
3. Восстанавливать оба значения после повторного запуска
4. Отображать сохраненные данные на экране
5. Реализовать удаление сохраненных данных

Для хранения возраста используйте методы `putInt()` и `getInt()`, подсказка:

| Метод | Тип данных | Пример |
|---|---|---|
| `putString()` | `String` | `"Иван"` |
| `putInt()` | `int` | `5` |
| `putBoolean()` | `boolean` | `true` |
| `putFloat()` | `float` | `18.5f` |
| `putLong()` | `long` | `100000L` |

## Дополнительно (по желанию)

1. **Запоминание города.** Добавьте поле для ввода города и реализуйте его сохранение и восстановление после перезапуска приложения
2. **Счетчик запусков.** Реализуйте подсчет количества запусков приложения с сохранением результата
3. **Проверка ввода.** Запретите сохранение пустого имени и выводите сообщение об ошибке